

REPLAY MEMORY AS AN EMPIRICAL MDP: COMBINING CONSERVATIVE ESTIMATION WITH EXPERIENCE REPLAY

Presenter: Hongming Zhang

Date: 2022.10.05

Experience replay stores transitions in a replay memory for repeated use. In this work, we try to make experience replay more efficient.

- We treat the replay memory as an *empirical Replay Memory MDP (RM-MDP)*.
- We *learn a conservative value estimate* by solving the RM-MDP and *design value and policy regularizers* based on this conservative estimate.
- Our method, *Conservative Estimation with Experience Replay (CEER)*, combines the two regularizers with experience replay, improves sample efficiency by a large margin, especially when the memory density is high.

- Experience replay plays an important role of improving sample efficiency in RL. Current works try to make experience replay more efficient from different directions:
 - 1) Reweighted Experience Replay: PER, ReF-ER, AER, DisCor.
 - 2) Experience Replay with Episodic RL: EMDQN, ERLAM.
 - 3) Experience Replay with Reverse Update: EBU, TER.
- Can we further improve the use of the replay memory by **considering all existing transitions as a whole?**

Algorithm

- Constructing RM-MDP using directed graph
- Solving RM-MDP
- Regularizing the learning with value and policy regularizers

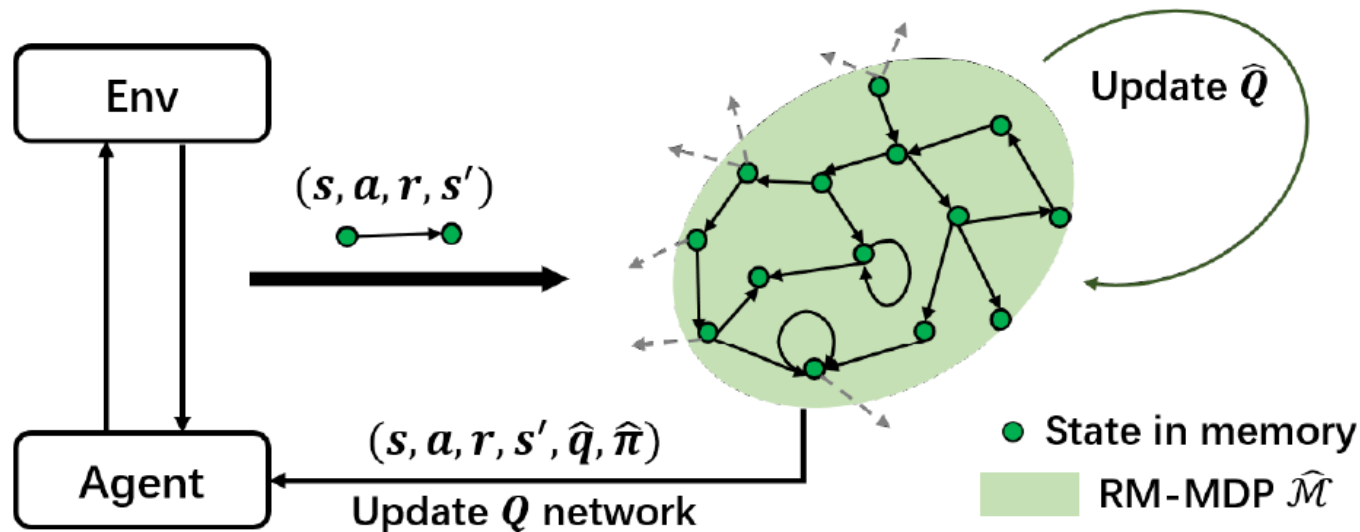


Figure 1: An overview of our method. Transitions from the interaction with environment constructs RM-MDP, which is part of the environment MDP. Green circles denote states in the memory. The solid lines denote existing transitions, and dashed lines denote transitions not in the memory. We obtain value and policy estimates from RM-MDP, which will be provided to regularize the learning.

Constructing RM-MDP



- Markov Decision Process (MDP)

MDP of the environment/task: $\mathcal{M} = (\mathcal{S}, \mathcal{A}, R, P, \rho_0, \gamma)$

RM-MDP: $\hat{\mathcal{M}} = (\mathcal{S}, \mathcal{A}, R, \hat{P}, \rho_0, \gamma)$

- Data structure

Construct the replay memory as a directed graph $\mathcal{G} = (V, E)$:

$$V = \{\phi(s) | (s, \hat{Q}(s, \cdot))\}$$
$$E = \{\phi(s) \xrightarrow{a} \phi(s') | (a, r, N(s, a, s'))\}$$

ϕ is a representation function, now it is only a hashing function serving as keys for fast query. $N(s, a, s')$ is visit count.

We use Q and $\hat{Q}$ to denote the estimation from network and RM-MDP respectively.

Constructing RM-MDP



- Property of the graphed memory

Fixed memory size that stores most recent experiences (counted by vertices).
Will **merge the same states** under the representation function ϕ .

- Update rule

When a new transition (s, a, r, s') comes,

If $\phi(s)$ in V :

Update the visit count: $N(s, a, s') \leftarrow N(s, a, s') + 1$

Else:

Create a new vertex and edge,

$V = \{\phi(s) | (s, \hat{Q}(s, \cdot))\}$, $E = \{\phi(s) \xrightarrow{a} \phi(s') | (a, r, N(s, a, s'))\}$.

Initialize $\hat{Q}(s, a) = 0$, $N(s, a, s') = 1$

We **estimate $\hat{P}$** for transition (s, a, s') with visit counts:

$$\hat{p}(s' | s, a) = \frac{N(s, a, s')}{\sum_{s'} N(s, a, s')}$$

Solving RM-MDP



With the graph, we have all elements of RM-MDP.
We use value iteration to update $\hat{Q}$:

$$\hat{Q} \leftarrow \sum_{s'} \hat{p}(s'|s, a) [r + \gamma \max_{a': \beta(a'|s') > 0} \hat{Q}(s', a')]$$

β is the behavior policy that collects data from the environment. We have $\hat{p}(s'|s, a) = 0$ if $\beta(a|s) = 0$.

The learning goal is to maximize the cumulative rewards but **limited within the replay memory**, which returns a conservative policy with best possible performance.

Sampling-based update: we use the sampled batch to update $\hat{Q}$ before updating Q -network. (No extra sampling computation.)

Episode-based update: when an episode ends, we update $\hat{Q}$ along the episode in reversed order. (Efficient especially with sparse/delayed reward environments.)

Regularizing DQN

- Value regularizer

Target Q value estimated by Q -network:

$$q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Target Q value estimated from RM-MDP:

$$\hat{q}(s, a) = \sum_{s'} \hat{p}(s'|s, a) [r + \gamma \max_{a': \beta(a'|s') > 0} \hat{q}(s', a')]$$

Target value:

$$q_{\text{target}} = (1 - \alpha)q(s, a) + \alpha \hat{q}(s, a)$$

$\alpha \in [0, 1]$ is a parameter.

Value loss:

$$\mathcal{L}_{TD} = \frac{1}{|\mathcal{D}|} \sum_{(s, a) \in \mathcal{D}} \left(q_{\text{target}}(s, a) - Q(s, a) \right)^2$$

$\mathcal{D}$ is a batch of experiences.

Regularizing DQN



- Policy regularizer

Boltzmann policy derive from $\hat{Q}$:

$$\hat{\pi}(s) = \text{softmax}_{\tau, \beta(a|s) > 0}(\hat{Q}(s, \cdot))$$

Boltzmann policy derive from Q :

$$\pi(s) = \text{softmax}_{\tau, \beta(a|s) > 0}(Q(s, \cdot))$$

τ is the softmax temperature.

Policy loss based on KL divergence:

$$\mathcal{L}_{policy} = \frac{\lambda}{|\mathcal{D}|} \sum_{s \in \mathcal{D}} D_{KL}(\hat{\pi}(s) | \pi(s))$$

$\lambda > 0$ is a parameter, $\mathcal{D}$ is a batch of experiences.

Combining the two regularizers, the update loss is:

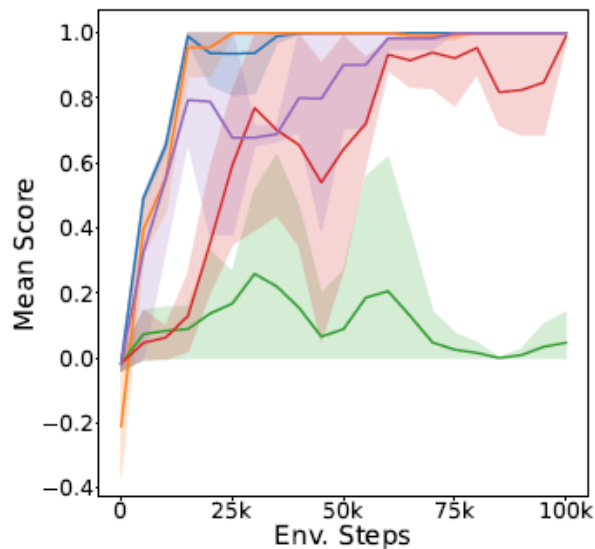
$$\mathcal{L} = \mathcal{L}_{TD} + \mathcal{L}_{policy}$$

Algorithm 1 Combine Conservative Estimation with Experience Replay (CEER)

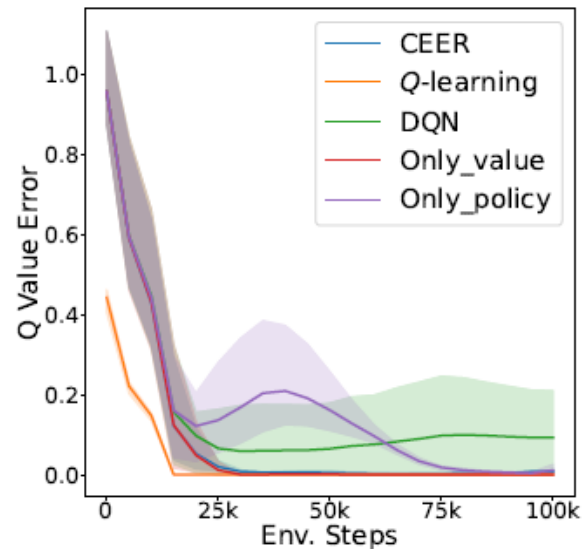
- 1: Initialize replay memory $\hat{M}$ with fixed memory size
 - 2: Initialize action value network Q with weights θ , target network $\bar{Q}$ with weights $\bar{\theta} \leftarrow \theta$
 - 3: Initialize the environment $s_0 \leftarrow Env$
 - 4: **for** environments step $t = 0$ **to** T **do**
 - 5: Select an action a_t by ϵ -greedy with respect to $Q(s_t, \cdot; \theta)$
 - 6: Execute a_t in Env and get r_t, s_{t+1}
 - 7: Store the transition (s_t, a_t, r_t, s_{t+1}) in $\hat{M}$ and update visit count $N(s_t, a_t, s_{t+1})$
 - 8: Sample random minibatch of transitions from $\hat{M}$
 - 9: Update $\hat{Q}(s_t, a_t)$ via equation 4 and store the result to $\hat{M}$
 - 10: Update $Q(s_t, a_t; \theta)$ by minimizing $\mathcal{L}$ via equation 8
 - 11: Update $\bar{\theta} \leftarrow \theta$ every C steps
 - 12: **end for**
-

Experiments

- Toy Example on Cliffworld



(a) Learning curves.



(b) Q value error.

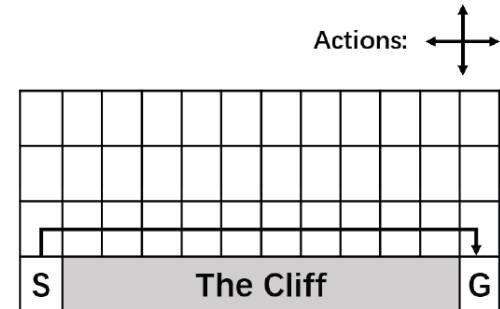


Figure 2: (a) The learning curve of each algorithm. CEER achieves comparable learning speed with Q -learning. The policy regularizer contributes more on policy learning. (b) The value error compared with optimal Q value during learning. The value regularizer contributes more on reducing the value error.

Experiments

- Overall performance on Sokoban, MiniGrid and MinAtar environments.

Table 1: Overall performance on Sokoban, MiniGrid and MinAtar environments. Numbers in bold represent the method that achieves the highest final scores.

Environment		DQN	DDQN	PER	EBU	DisCor	TER	CEER
Sokoban	Push-5×5-1	10.82	10.78	10.81	7.68	10.76	10.82	10.82
	Push-5×5-2	10.52	8.72	9.61	6.12	8.39	11.10	11.62
	Push-6×6-1	9.26	0.01	0.33	-3.55	-0.04	8.48	10.73
	Push-6×6-2	-1.93	-9.71	-9.81	-11.59	-10.43	-10.40	10.54
	Push-6×6-3	-10.06	-10.03	-10.27	-11.71	-10.56	-11.80	7.65
	Push-7×7-1	5.78	-5.60	-4.61	-6.39	-4.14	-0.69	10.65
	Push-7×7-2	-2.96	-11.76	-11.60	-11.86	-11.81	-11.90	9.97
MiniGrid	DoorKey	1.19	3.43	3.45	-3.60	-0.63	-3.60	3.50
	Unlock	-0.87	2.72	2.74	-2.88	-2.72	2.59	2.80
	RedBlueDoors	5.40	4.67	7.06	-7.22	5.77	6.70	7.09
	SimpleCrossing-Easy	3.08	3.09	3.07	2.98	3.02	3.08	3.11
	SimpleCrossing-Hard	3.03	3.02	3.02	-2.85	2.70	3.01	3.09
	LavaCrossing-Easy	0.58	2.24	1.52	-3.00	-1.13	-2.28	3.10
	LavaCrossing-Hard	-3.34	-3.37	-3.38	-3.31	-3.38	-3.09	3.05
MinAtar	Asterix	19.82	20.1	25.8	26.0	31.19	3.07	19.82
	Breakout	13.57	16.25	17.76	19.97	17.14	12.73	42.53
	Freeway	57.85	57.66	57.79	59.07	56.85	22.08	58.77
	Seaquest	14.73	18.57	17.48	12.65	16.69	1.62	17.96
	SpaceInvaders	50.71	51.15	55.09	71.83	57.85	45.29	73.43

Experiments



- Relationship between performance improvement and conservative estimation
- Average Relative Performance Improvement (**ARPI**):

$$ARPI(A|B) = \frac{1}{K} \sum_{k=1}^K \frac{R_k(A) - R_k(B)}{R_{\max} - R_k(B)} \leq 1$$

$R_k(\cdot)$ is the cumulative reward at evaluating iteration k , $R_{\max}$ is the maximum cumulative reward that the two methods achieved during the learning process.

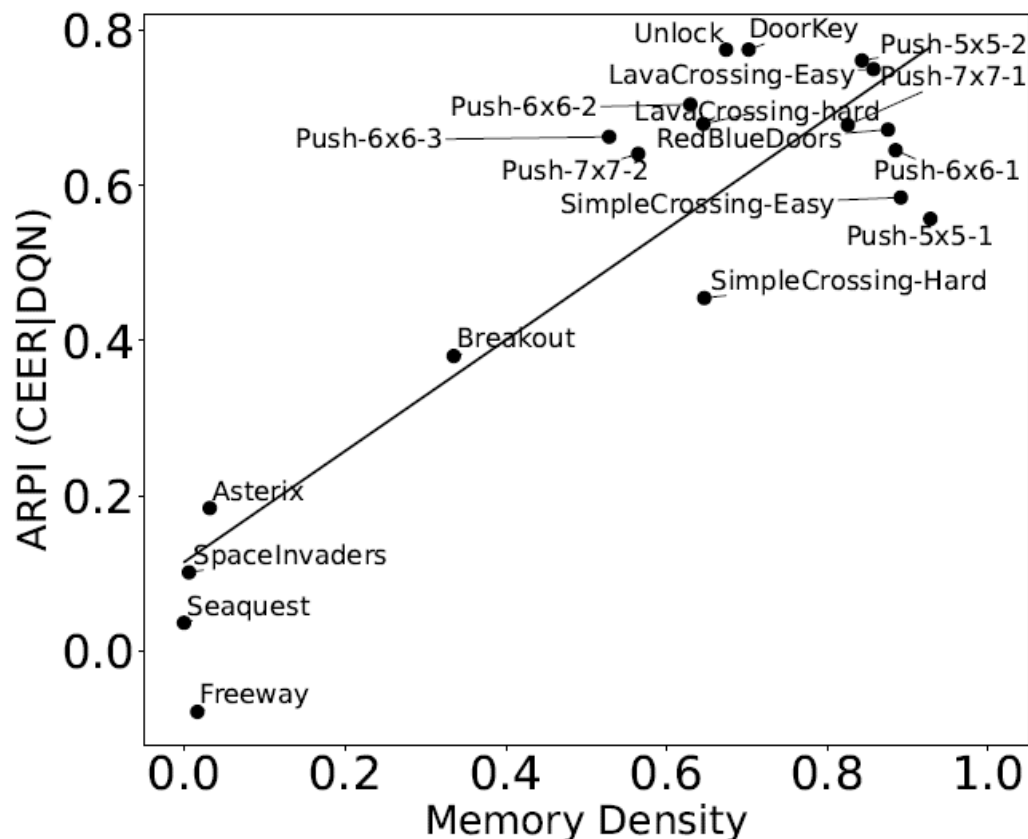
- Memory **Density**:

$$\text{Density}(\hat{M}) = \frac{D_{\text{out}}(\hat{M}) - 1}{|\mathcal{A}| - 1} \in [0,1]$$

$D_{\text{out}}(s)$ is the out-degree of the vertex at state s , $D_{\text{out}}(\hat{M}) = \frac{1}{|\hat{M}|} \sum_{s \in \hat{M}} D_{\text{out}}(s)$, $|\mathcal{A}|$ is the size of action space.

Experiments

- Relationship between performance improvement and conservative estimation



Pearson Correlation Coefficient (PCC):

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1,1]$$

$$\rho_{\text{Density,ARPI}} = 0.881$$

Figure 3: Relationship between memory density and the performance improvement. We compare CEER with DQN. The memory density is the average value during learning. The oblique line shows the result of linear regression. There is a strong correlation between memory density and ARPI.

Experiments

- Ablation studies of the two regularizers across environments with different memory densities

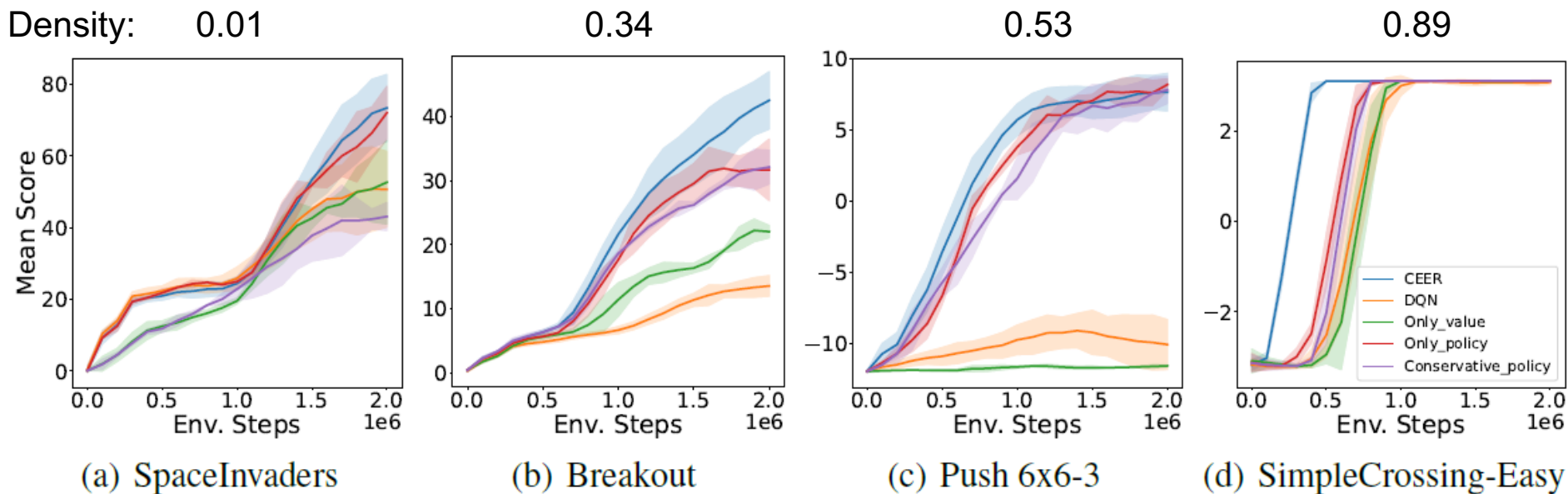


Figure 4: Ablation studies of the two regularizers across environments with different memory densities. **Only_value** and **Only_policy** mean only use value regularizer or policy regularizer, **Conservative_policy** is the policy derived from RM-MDP by supervised learning. CEER combines both regularizers.

Conclusion



- This paper proposes to consider the replay memory as an **empirical MDP (RM-MDP)**. Solving this RM-MDP combines existing experiences in the replay memory into a **conservative estimate**.
- **Two regularizers** based on the estimation are designed to accelerate online DRL (CEER).
- CEER shows the effectiveness in many environments and observe a strong correlation between the performance improvement and memory density of RM-MDP.

Future work



- (1) Design **representation function ϕ** to merge similar states.
 - Learn a representation vector with contrastive learning or like MuZero style.
- (2) Extend CEER to environments with **continuous action space**.
 - Discretize the action space, then build the graph.
 - Use offline RL method by sampling.



THANKS